

ARIMA Model Testing

```
In [1]: import itertools
import logging
import os
import time
import warnings
from concurrent.futures import ThreadPoolExecutor

import matplotlib.pyplot as plt
import mlflow
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
from dotenv import load_dotenv
from joblib import Parallel, delayed, parallel_config
from scipy import stats
from sklearn.exceptions import UndefinedMetricWarning
from sklearn.metrics import r2_score
from statsmodels.tsa.statespace.sarimax import SARIMAX
from tqdm import tqdm

load_dotenv()

ROOT_PATH = os.path.dirname(os.getcwd())
DATA_FOLDER_PATH = os.path.join(ROOT_PATH, 'data', 'processed')

COVID_START = pd.Timestamp('2020-04-01') # first pandemic month: April 2020
COVID_END = pd.Timestamp('2021-12-31')
POST_COVID_START = pd.Timestamp('2022-01-01')
COVID_RECOVERY_END = pd.Timestamp('2022-06-30')

sns.set_palette('colorblind')
logging.getLogger('mlflow').setLevel(logging.ERROR)
warnings.filterwarnings('ignore', category=UndefinedMetricWarning)
warnings.filterwarnings('ignore', category=FutureWarning, module='mlflow')
mlflow.set_tracking_uri(os.getenv('MLFLOW_TRACKING_URI'))
EXPERIMENT_NAME = 'Brazil Tourism Forecasting ARIMA'
experiment = mlflow.set_experiment(EXPERIMENT_NAME)
```

Target Transformations

Three target transformations are compared in the grid. The model is always evaluated on the **original scale**: forecasts are inverse-transformed back to arrival counts before computing the validation metrics.

Option	Transform	Inverse
none	identity (raw arrival counts)	identity

Option	Transform	Inverse
log	log(y)	exp($\hat{y}$)
auto_stationary	AutoStationaryTransformer (AutoML)	exact inverse_transform of the fitted pipeline

auto_stationary comes from the companion repository of:

Joseph, M. & Tackes, J. *Modern Time Series Forecasting with Python*, 2nd ed. (Packt, 2024) — [PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E](#), MIT License

During `fit` (`src/transforms/target_transformations.py`) it runs the tests explored in notebook 3 and adds a step only when the test asks for it:

1. `check_trend` (Mann-Kendall). If a trend is found → `DetrendingTransformer` (linear).
2. `check_seasonality` at `m = 12`. If seasonal → `DeseasonalizingTransformer` (period averages).
3. `check_heteroscedasticity` (White). If heteroscedastic → `AddMTTransformer` when needed, then `BoxCoxTransformer` with Guerrero λ .

The transformer is fitted separately on each training window.

The repository is expected as a sibling folder of this project (override with the `MTSF_REPO_PATH` environment variable).

```
In [2]: import sys

# Helpers credited to: Joseph, M. & Tackes, J. "Modern Time Series Forecasting with
# Python", 2nd ed. (Packt, 2024) – MIT License
# github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E
MTSF_REPO_PATH = os.getenv(
    'MTSF_REPO_PATH',
    os.path.join(
        os.path.dirname(ROOT_PATH), 'Modern-Time-Series-Forecasting-with-Python-2E'
    ),
)
if MTSF_REPO_PATH not in sys.path:
    sys.path.insert(0, MTSF_REPO_PATH)

from src.transforms.target_transformations import ( # noqa: E402
    AutoStationaryTransformer,
    BoxCoxTransformer,
)
from src.utils.ts_utils import forecast_bias, metrics_adapter # noqa: E402

SEASONAL_PERIOD = 12
FREQ = 'ME'
```

```

def fit_auto_stationary(train_series):
    """Fit a fresh AutoStationaryTransformer (Mann-Kendall trend, Guerrero Box-Cox)
    # fresh dicts every fit: the transformer mutates its parameter dicts in place
    transformer = AutoStationaryTransformer(
        confidence=0.05,
        seasonal_period=SEASONAL_PERIOD,
        trend_check_params={'mann_kendall': True},
        detrender_params={'degree': 1},
        deseasonalizer_params={},
        box_cox_params={'optimization': 'guerrero'},
    )
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        return transformer.fit(train_series, freq=FREQ)

def describe_pipeline(transformer):
    steps = [
        type(tr).__name__.replace('Transformer', '') for tr in transformer._pipeline
    ]
    return ' → '.join(steps) if steps else '(none)'

class IdentityTarget:
    """No target transformation."""

    @staticmethod
    def transform(y):
        return y

    @staticmethod
    def inverse_transform(y):
        return y

class LogTarget:
    """Natural log of the arrival counts; inverse is exp."""

    @staticmethod
    def transform(y):
        return np.log(y)

    @staticmethod
    def inverse_transform(y):
        return np.exp(y)

def fit_target_transform(name, train_series):
    """Return a fitted transformer exposing `transform` / `inverse_transform`."""
    if name == 'none':
        return IdentityTarget()
    if name == 'log':
        return LogTarget()
    if name == 'auto_stationary':
        return fit_auto_stationary(train_series)
    raise ValueError(f'unknown target transform: {name}')

```

Data Preparation

Several training data selections are evaluated in the grid search:

- `last_{14,12,10,8,6,4}y_covid_exog` — last N years with a 0/1 COVID regressor (`is_covid`) to absorb the structural break
- `last_{14,12,10,8,6,4}y_covid_recovery_exog` — same windows but with **two** regressors: `is_covid` (Apr 2020–Dec 2021) **and** `is_covid_recovery` (Jan 2022–Jun 2022), the rebound period immediately after the pandemic
- `post_covid` — only post-COVID data (2022 onwards); no exog needed, cleaner signal
- `post_covid_recovery_exog` — same post-COVID window with `is_covid_recovery` (Jan 2022–Jun 2022) to explicitly model the rebound year within that short series

Every selection is crossed with the three target transformations (`none`, `log`, `auto_stationary`). The `AutoStationaryTransformer` is fitted on each training window only (the raw arrival counts).

The data is **train + validation** (`train.parquet` and `val.parquet`, notebook 2). The test set is never read here: it is kept for the final notebook, which refits the selected model and evaluates it once. Every date below is derived from the data, so the notebook follows it when new data is added.

```
In [3]: def load_monthly(file_name):
        raw = pd.read_parquet(os.path.join(DATA_FOLDER_PATH, file_name))
        return raw.set_index('date')[['arrival_count']].resample('ME').sum()

train_monthly = load_monthly('train.parquet')
val_monthly = load_monthly('val.parquet')
TRAIN_END = train_monthly.index[-1]
HORIZON = len(val_monthly)
assert val_monthly.index[0] == TRAIN_END + pd.offsets.MonthEnd(1)

# train + validation: every validation origin is cut from this history (never the t
full_monthly = pd.concat([train_monthly, val_monthly])
dates = full_monthly.index.to_series()
full_monthly['is_covid'] = dates.between(COVID_START, COVID_END).astype(float)
full_monthly['is_covid_recovery'] = dates.between(
    POST_COVID_START, COVID_RECOVERY_END
).astype(float)
train_monthly = full_monthly.loc[:TRAIN_END]
val_monthly = full_monthly.loc[TRAIN_END + pd.offsets.MonthEnd(1) :]

window_years = [14, 12, 10, 8, 6, 4]

print(
    f'Train + validation: {full_monthly.index[0]:%b %Y} → '
    f'{full_monthly.index[-1]:%b %Y} ({len(full_monthly)} obs)'
```

```

)
print(f'Train          : up to {TRAIN_END:%b %Y} ({len(train_monthly)} obs)')
print(
    f'Validation year  : {val_monthly.index[0]:%b %Y} → '
    f'{val_monthly.index[-1]:%b %Y} ({HORIZON} obs)'
)

```

```

Train + validation: Jan 1989 → Aug 2025 (440 obs)
Train              : up to Aug 2024 (428 obs)
Validation year    : Sep 2024 → Aug 2025 (12 obs)

```

Validation Origins (Repeated Holdout)

A single 12-month validation year can pick a model that only fits the quirks of that year, especially on a non-stationary series (Joseph & Tackes, ch. 19). The model is therefore scored on **several non-overlapping 12-month validation windows**, each with its own origin (the last month the model is trained on), a repeated holdout without overlap:

- **Horizon.** Every window has `HORIZON` = 12 months, the real use of the model.
- **Post-COVID origins** (`N_POST_COVID_ORIGINS` = 2). The end of training (its validation window is `val.parquet`) and one year earlier: always the **two most recent years** of the data. Their windows start after the pandemic.
- **Pre-COVID origins** (`N_PRE_COVID_ORIGINS` = 2). Counted back from the end of training one year at a time, the two most recent origins whose validation window ends **before** `COVID_START` . An origin one year later would validate on the 2020 collapse.
- **Holdout, not cross-validation.** The COVID flags are exogenous variables, so a holdout per origin is safer than a shuffled or blocked cross-validation (ch. 19).

Only the two counts are fixed. Every date is derived from the end of training, so when new data is added the post-COVID origins move to the newest years, and the pre-COVID origins keep the same calendar month as the end of training (as close to the pandemic as possible). Every origin sees only the history up to that origin: the training windows (`last_Ny`) are counted back from it and the target transformations are fitted on it.

Two consequences of the pre-COVID origins:

- Before the pandemic the COVID flags are all zero, so `*_covid_exog` and `*_covid_recovery_exog` are the same model there (fitted once, scored for both).
- The `post_covid*` selections only exist at the post-COVID origins.

```

In [4]: N_PRE_COVID_ORIGINS = 2
        N_POST_COVID_ORIGINS = 2

def validation_window(origin):
    """First and last month forecast from `origin` (the last month the model sees).
    return origin + pd.offsets.MonthEnd(1), origin + pd.offsets.MonthEnd(HORIZON)

```

```

# candidate origins: the end of training, then one year back at a time
candidates = [TRAIN_END] + [
    TRAIN_END - pd.offsets.MonthEnd(12 * k) for k in range(1, len(train_monthly) //
]
post_covid_origins = [o for o in candidates if validation_window(o)[0] > COVID_END]
                    :N_POST_COVID_ORIGINS
]
pre_covid_origins = [o for o in candidates if validation_window(o)[1] < COVID_START]
                    :N_PRE_COVID_ORIGINS
]
ORIGINS = sorted(pre_covid_origins + post_covid_origins)
LATEST_ORIGIN = TRAIN_END # its validation window is val.parquet
assert LATEST_ORIGIN in ORIGINS

def origin_label(origin):
    return f'{origin:%Y-%m}'

def val_actual_at(origin):
    start, end = validation_window(origin)
    return full_monthly.loc[start:end, 'arrival_count']

pd.DataFrame([
    {
        'origin': origin_label(origin),
        'phase': 'pre-COVID' if origin < COVID_START else 'post-COVID',
        'validation window': f'{start:%b %Y} → {end:%b %Y}',
        'history months': len(full_monthly.loc[:origin]),
        'validation arrivals': f'{val_actual_at(origin).sum():.0f}',
    }
    for origin in ORIGINS
    for start, end in [validation_window(origin)]
])

```

Out[4]:

	origin	phase	validation window	history months	validation arrivals
0	2017-08	pre-COVID	Sep 2017 → Aug 2018	344	6,655,969
1	2018-08	pre-COVID	Sep 2018 → Aug 2019	356	6,408,872
2	2023-08	post-COVID	Sep 2023 → Aug 2024	416	6,340,127
3	2024-08	post-COVID	Sep 2024 → Aug 2025	428	8,849,640

Evaluation Metrics and Baselines

Every model is scored at every origin on the original scale (arrival counts), with the same six metrics as notebook 7:

| Metric | Definition | Reading | |---|---|---| | **Seasonal MASE** | MAE / MAE(seasonal naive, in-sample) | **Selection metric.** Below 1: better than "same month last year" was

on the history || **RMSE** | $\sqrt{\text{mean}((y - \hat{y})^2)}$ | Penalizes large misses || **MAE** | $\text{mean}(|y - \hat{y}|)$ | Typical monthly miss, in arrivals || **MAPE** | $\text{mean}(|y - \hat{y}| / y) \times 100$ | Typical miss in % || **R²** | $1 - \text{SSE} / \text{SST}$ | Share of the validation-year variation explained || **Forecast Bias** | $(\Sigma y - \Sigma \hat{y}) / \Sigma y \times 100$ | Positive = the model **under**-forecasts the year |

Why seasonal MASE. RMSE and MAE are in arrivals, so an origin with a higher level (or the +40% jump of 2025) would dominate an average across origins. MASE divides each origin's MAE by the in-sample MAE of the seasonal naive forecast $y_{\{t-12\}}$ on that origin's history, which makes the origins comparable. The seasonal version ($m = 12$) is used instead of the book's `mase` ($m = 1$), because the one-step naive error of a strongly seasonal series mostly measures the seasonality. Pairs that touch the pandemic (April 2020 to December 2022, collapse and rebound) are left out of the scale, so they do not inflate it at the post-COVID origins.

Ranking. Configurations are ranked by the **mean seasonal MASE over the origins** (`mase_mean`). The worst origin (`mase_worst`) shows how stable a configuration is, and `mase_post` compares every configuration on the two post-COVID origins only, the only ones where the `post_covid*` selections exist. RMSE, MAE, MAPE, R², MASE and Forecast Bias of the latest origin (the validation year, `val.parquet`) are kept for reading the forecasts. Forecast Bias uses the book's `forecast_bias` with `metrics_adapter` (`src/utils/ts_utils.py`).

Baselines (chapter 8, as in notebook 7), at every origin:

- **naive:** every validation month repeats the last training month.
- **seasonal naive:** every validation month repeats the same month one year earlier.

A SARIMAX that does not beat the seasonal naive has not learned anything beyond "same as last year".

```
In [5]: METRICS = ['rmse', 'mae', 'mape', 'r2', 'mase', 'forecast_bias']
AGG_METRICS = ['mase_mean', 'mase_worst', 'mase_post', 'n_origins']
METRIC_LABELS = {
    'mase_mean': f'Mean seasonal MASE ({len(ORIGINS)} origins)',
    'mase_worst': 'Worst-origin seasonal MASE',
    'rmse': 'RMSE (validation year)',
    'mae': 'MAE (validation year)',
    'mape': 'MAPE % (validation year)',
    'r2': 'R2 (validation year)',
    'mase': 'Seasonal MASE (validation year)',
    'forecast_bias': 'Forecast Bias % (validation year)',
}
# seasonal differences distorted by the pandemic collapse and rebound
MASE_EXCLUDED = (COVID_START, pd.Timestamp('2022-12-31'))

def seasonal_mase_scale(history):
    """In-sample MAE of the seasonal naive (y_t - y_{t-12}), pandemic pairs exclude
    diffs = (history - history.shift(SEASONAL_PERIOD)).abs()
```

```

in_pandemic = history.index.to_series().between(*MASE_EXCLUDED)
touches = in_pandemic | in_pandemic.shift(SEASONAL_PERIOD, fill_value=False)
return float(diffs[~touches.to_numpy()].mean())

MASE_SCALES = {
    origin: seasonal_mase_scale(full_monthly.loc[:origin, 'arrival_count'])
    for origin in ORIGINS
}

def score(actual, forecast, origin):
    """Metrics of one origin on the original scale (arrival counts)."""
    forecast = pd.Series(np.asarray(forecast, dtype=float), index=actual.index)
    errors = actual - forecast
    mae = float(np.mean(np.abs(errors)))
    return {
        'rmse': float(np.sqrt(np.mean(errors**2))),
        'mae': mae,
        'mape': float(np.mean(np.abs(errors / actual)) * 100),
        'r2': float(r2_score(actual, forecast)),
        'mase': mae / MASE_SCALES[origin],
        'forecast_bias': float(metrics_adapter(forecast_bias, actual, forecast)),
    }

def aggregate(per_origin):
    """Combine the per-origin metrics of one configuration ({origin: metrics})."""
    mase = pd.Series({origin: m['mase'] for origin, m in per_origin.items()})
    post = mase[mase.index.isin(post_covid_origins)]
    latest = per_origin.get(LATEST_ORIGIN, {})
    return {
        'mase_mean': float(mase.mean()),
        'mase_worst': float(mase.max()),
        'mase_post': float(post.mean()) if len(post) else np.nan,
        'n_origins': len(mase),
        # the latest origin forecasts the validation year (val.parquet)
        **{key: latest.get(key, np.nan) for key in METRICS},
    }

def baseline_forecasts(origin):
    history = full_monthly.loc[:origin, 'arrival_count']
    dates = val_actual_at(origin).index
    return {
        'naive': pd.Series(history.iloc[-1], index=dates),
        'seasonal_naive': pd.Series(
            history.reindex(dates - pd.offsets.MonthEnd(12)).to_numpy(), index=dates
        ),
    }

baseline_scores = {'naive': {}, 'seasonal_naive': {}}
for origin in ORIGINS:
    for name, forecast in baseline_forecasts(origin).items():
        baseline_scores[name][origin] = score(val_actual_at(origin), forecast, origin)

```

```

baselines = pd.DataFrame({
    name: aggregate(per_origin) for name, per_origin in baseline_scores.items()
}).T
BASELINE_MASE = float(baselines.loc['seasonal_naive', 'mase_mean'])
val_actual = val_monthly['arrival_count']

display(
    pd.DataFrame({
        name: {origin_label(o): m['mase'] for o, m in per_origin.items()}
        for name, per_origin in baseline_scores.items()
    }).rename_axis('seasonal MASE by origin')
)
baselines[[*AGG_METRICS, *METRICS]].astype(float).round(3)

```

	naive	seasonal_naive
seasonal MASE by origin		
2017-08	3.325103	0.680184
2018-08	2.441616	0.977196
2023-08	3.393431	1.462635
2024-08	6.148383	4.023995

Out[5]:

	mase_mean	mase_worst	mase_post	n_origins	rmse	mae	mi
naive	3.827	6.148	4.771	4.0	460440.495	319530.000	33.
seasonal_naive	1.786	4.024	2.743	4.0	255353.116	209126.083	26.

Model Selection Grid

Grid dimensions:

- **SARIMAX orders:** $p, q \in \{0, 1, 2\}$, $d \in \{0, 1\}$, $P, Q \in \{0, 1\}$, $D \in \{0, 1\}$, $s = 12 \rightarrow 144$ configs. The differencing orders are **searched**, not fixed, and independently of the target transformation. This way, the grid shows directly whether a transformation (e.g. the AutoML detrend/deseasonalize) replaces differencing or still needs it.
- **Target transformation:** `none`, `log`, `auto_stationary` $\rightarrow 3$ options
- **Training data selection:** `last_14y/12y/10y/8y/6y/4y_covid_exog`, `last_14y/12y/10y/8y/6y/4y_covid_recovery_exog`, `post_covid`, or `post_covid_recovery_exog` $\rightarrow 14$ options
- **Configurations:** 6,048 (order, target transformation, data selection), each scored at every origin where its data selection exists.

Execution. The grid runs in two phases:

1. **Fit in parallel.** Each task (origin, data selection, transformation, order) is fitted and scored in a separate process with `joblib` (`N_JOBS` workers, one BLAS thread each to avoid oversubscription). Workers never touch MLflow; every task catches its own errors and returns them as a failed result. The two exogenous variants of a pre-COVID origin are fitted once.
2. **Aggregate and log from the main process.** The per-origin scores of a configuration are combined (`aggregate`). Only the configurations that converged at every origin **and** have a mean seasonal MASE below `MLFLOW_MAX_MASE` = 1.5 are logged to MLflow (as in notebook 8), one run each: the six choices as parameters, the aggregated metrics, the validation-year metrics and the seasonal MASE of each origin (`mase_origin_YYYY_MM`). The rest stay in `configs_df` only; a configuration that fails at any of its origins is left out of the ranking. Runs are logged from `N_LOG_THREADS` threads, because the cost is the HTTP round-trips to the server, not CPU.

AIC/BIC (of the latest origin) are computed on the transformed scale and depend on the differencing, so they are only comparable *within* the same data selection, transformation and (d, D) . Models are ranked across the grid by mean seasonal MASE.

```
In [6]: s = 12

p_range = range(0, 3)
d_range = range(0, 2)
q_range = range(0, 3)
P_range = range(0, 2)
D_range = range(0, 2)
Q_range = range(0, 2)

sarimax_grid = list(
    itertools.product(p_range, d_range, q_range, P_range, D_range, Q_range)
)

TARGET_TRANSFORMS = ['none', 'log', 'auto_stationary']
EXOG_VARIANTS = {
    'covid_exog': ['is_covid'],
    'covid_recovery_exog': ['is_covid', 'is_covid_recovery'],
}

# — training data selections of one origin (raw target + exogenous regressors) —
def selection(frame, val, exog_columns):
    return {
        'raw_series': frame['arrival_count'],
        'exog_train': frame[exog_columns] if exog_columns else None,
        'exog_val': val[exog_columns] if exog_columns else None,
        'val_actual': val['arrival_count'],
    }

def data_selections_at(origin):
```

```

"""Selections that exist at `origin`; {'alias_of': key} marks a duplicate model
history = full_monthly.loc[:origin]
start, end = validation_window(origin)
val = full_monthly.loc[start:end]
has_covid = origin >= COVID_START
selections = {}
for years in window_years:
    window = history[history.index >= origin - pd.DateOffset(years=years)]
    base_key = f'last_{years}y_covid_exog'
    for suffix, exog_columns in EXOG_VARIANTS.items():
        key = f'last_{years}y_{suffix}'
        if has_covid:
            selections[key] = selection(window, val, exog_columns)
        elif key == base_key:
            # before the pandemic the flags are all zero: no regressor
            selections[key] = selection(window, val, None)
        else:
            selections[key] = {'alias_of': base_key}
if origin > POST_COVID_START:
    post = history[history.index >= POST_COVID_START]
    selections['post_covid'] = selection(post, val, None)
    # Post-COVID window with recovery flag to model the Jan-Jun 2022 rebound
    selections['post_covid_recovery_exog'] = selection(
        post, val, ['is_covid_recovery']
    )
return selections

selections_by_origin = {origin: data_selections_at(origin) for origin in ORIGINS}

# — (origin × data selection × target transformation) configs —————
def make_config(selection, transform_name):
    """Fit the target transformation on the training window and store the pieces."""
    transformer = fit_target_transform(transform_name, selection['raw_series'])
    is_auto = transform_name == 'auto_stationary'
    box_cox = (
        [tr for tr in transformer._pipeline if isinstance(tr, BoxCoxTransformer)]
        if is_auto
        else []
    )
    return {
        **selection,
        'target_transform': transform_name,
        'series': transformer.transform(selection['raw_series']),
        'transformer': transformer,
        'pipeline': describe_pipeline(transformer) if is_auto else transform_name,
        'boxcox_lambda': box_cox[0].boxcox_lambda if box_cox else None,
    }

model_configs = {
    (origin, data_key, transform_name): make_config(sel, transform_name)
    for origin, selections in selections_by_origin.items()
    for data_key, sel in selections.items()
    if 'alias_of' not in sel

```

```

    for transform_name in TARGET_TRANSFORMS
}
data_selections = selections_by_origin[LATEST_ORIGIN] # every selection exists here
expected_origins = {
    key: sum(key in selections_by_origin[origin] for origin in ORIGINS)
    for key in data_selections
}

# — what the AutoML pipeline chose for each training window (latest origin) —
transform_summary = pd.DataFrame([
    {
        'data_selection': data_key,
        'n_train': len(cfg['raw_series']),
        'pipeline': cfg['pipeline'],
        'trend (MK p)': round(cfg['transformer']._trend_check['p_value'], 4),
        'seasonal': cfg['transformer']._seasonality_check['seasonal'],
        'hetero (White p)': round(cfg['transformer']._hetero_check['lm_p_value'], 4),
        'boxcox_λ': None
        if cfg['boxcox_lambda'] is None
        else round(cfg['boxcox_lambda'], 3),
    }
    for (origin, data_key, transform_name), cfg in model_configs.items()
    if transform_name == 'auto_stationary' and origin == LATEST_ORIGIN
]).set_index('data_selection')

n_configs = len(sarimax_grid) * len(data_selections) * len(TARGET_TRANSFORMS)
print(f'SARIMAX configs      : {len(sarimax_grid)}')
print(f'Target transformations: {len(TARGET_TRANSFORMS)}')
print(f'Data selections      : {len(data_selections)}')
print(f'Configurations        : {n_configs}')
print(f'Origins                : {"", ".join(origin_label(o) for o in ORIGINS)}')
print(f'Fits (all origins)     : {len(sarimax_grid) * len(model_configs)}')
display(pd.Series(expected_origins, name='origins per data selection'))
transform_summary

```

```

SARIMAX configs      : 144
Target transformations: 3
Data selections      : 14
Configurations        : 6048
Origins                : 2017-08, 2018-08, 2023-08, 2024-08
Fits (all origins)     : 17280
last_14y_covid_exog      4
last_14y_covid_recovery_exog 4
last_12y_covid_exog      4
last_12y_covid_recovery_exog 4
last_10y_covid_exog      4
last_10y_covid_recovery_exog 4
last_8y_covid_exog       4
last_8y_covid_recovery_exog 4
last_6y_covid_exog       4
last_6y_covid_recovery_exog 4
last_4y_covid_exog       4
last_4y_covid_recovery_exog 4
post_covid                2
post_covid_recovery_exog  2
Name: origins per data selection, dtype: int64

```

Out[6]:

	n_train	pipeline	trend (MK p)	seasonal	hetero (White p)	boxcox_λ
data_selection						
last_14y_covid_exog	169	Detrending → Deseasonalizing → AddM → BoxCox	0.0061	True	0.0048	1.649
last_14y_covid_recovery_exog	169	Detrending → Deseasonalizing → AddM → BoxCox	0.0061	True	0.0048	1.649
last_12y_covid_exog	145	Detrending → Deseasonalizing → AddM → BoxCox	0.0017	True	0.0397	2.000
last_12y_covid_recovery_exog	145	Detrending → Deseasonalizing → AddM → BoxCox	0.0017	True	0.0397	2.000
last_10y_covid_exog	121	Detrending → Deseasonalizing → AddM → BoxCox	0.0041	True	0.0015	2.000
last_10y_covid_recovery_exog	121	Detrending → Deseasonalizing → AddM → BoxCox	0.0041	True	0.0015	2.000
last_8y_covid_exog	97	Deseasonalizing → AddM → BoxCox	0.0741	True	0.0045	2.000
last_8y_covid_recovery_exog	97	Deseasonalizing → AddM → BoxCox	0.0741	True	0.0045	2.000
last_6y_covid_exog	73	Deseasonalizing → AddM → BoxCox	0.1658	True	0.0152	1.688
last_6y_covid_recovery_exog	73	Deseasonalizing → AddM → BoxCox	0.1658	True	0.0152	1.688
last_4y_covid_exog	49	Deseasonalizing	0.0583	True	0.4822	NaN
last_4y_covid_recovery_exog	49	Deseasonalizing	0.0583	True	0.4822	NaN
post_covid	32	Deseasonalizing	0.2085	True	0.0900	NaN
post_covid_recovery_exog	32	Deseasonalizing	0.2085	True	0.0900	NaN

MLflow Setup

`mlflow.statsmodels.autolog` is **disabled**:

- It saved a pickled statsmodels model (plus a full `uv` environment export) for every run, which took about 90% of each run's time. A SARIMAX fit takes 0.02–0.5 s, while a run with autolog took about 5 s.
- The grid fits run in parallel worker processes, where autolog would not work.
- The `AutoStationaryTransformer` tests (ADF, White/OLS) call statsmodels `fit` internally, and autolog would log each of them as a separate run.

Each grid run logs its parameters and metrics explicitly. Only the **best** model is saved as a model artifact, at the end of the notebook (*Save the Best Model to MLflow*).

The tracking server runs in Docker (`docker compose up -d mlflow`, or `task mlflow`) and is reached through `MLFLOW_TRACKING_URI=http://localhost:5000` in `.env`.

```
In [7]: mlflow.statsmodels.autolog(disable=True)

print(f'Tracking URI: {mlflow.get_tracking_uri()}')
print(f'Experiment : {experiment.name} (id {experiment.experiment_id})')
```

```
Tracking URI: http://localhost:5000
Experiment : Brazil Tourism Forecasting ARIMA (id 1)
```

```
In [8]: # physical cores: SARIMAX fits are CPU-bound, hyper-threads add little
N_JOBS = max(1, (os.cpu_count() or 2) // 2)

def forecast_arrivals(fit, cfg):
    """Forecast the validation window on the transformed scale, back to arrivals."""
    index = cfg['val_actual'].index
    transformed = np.asarray(fit.forecast(steps=len(index), exog=cfg['exog_val']))
    forecast = cfg['transformer'].inverse_transform(pd.Series(transformed, index=index))
    return forecast.to_numpy()

def fit_one(origin, data_key, transform_name, cfg, order_spec):
    """Fit and score one grid point at one origin (worker process, no MLflow)."""
    p, d, q, P, D, Q = order_spec
    result = {
        'origin': origin,
        'data_selection': data_key,
        'target_transform': transform_name,
        'order': (p, d, q),
        'seasonal_order': (P, D, Q, s),
    }
    start = time.perf_counter()
    try:
        with warnings.catch_warnings():
```

```

warnings.simplefilter('ignore')
fit = SARIMAX(
    cfg['series'],
    exog=cfg['exog_train'],
    order=(p, d, q),
    seasonal_order=(P, D, Q, s),
    enforce_stationarity=False,
    enforce_invertibility=False,
).fit(dispatch=False)

forecast = forecast_arrivals(fit, cfg)
if not np.all(np.isfinite(forecast)):
    raise ValueError('non-finite forecast after inverse transform')
result.update({
    'aicc': fit.aicc,
    'aic': fit.aic,
    'bic': fit.bic,
    **score(cfg['val_actual'], forecast, origin),
    'error': None,
})
except Exception as exc:
    result['error'] = str(exc)[:250]
result['fit_seconds'] = time.perf_counter() - start
return result

# — phase 1: fit every (origin, grid point) in parallel —————
tasks = [
    (origin, data_key, transform_name, cfg, order_spec)
    for (origin, data_key, transform_name), cfg in model_configs.items()
    for order_spec in sarimax_grid
]
print(f'Fitting {len(tasks)} models on {N_JOBS} workers...')

grid_start = time.perf_counter()
with parallel_config(backend='loky', inner_max_num_threads=1):
    results = Parallel(n_jobs=N_JOBS, verbose=5)(
        delayed(fit_one)(*task) for task in tasks
    )
print(f'Fitting done in {time.perf_counter() - grid_start:.0f}s')

# before the pandemic both exogenous variants are the same model: copy its scores
fits = pd.DataFrame(results)
aliases = [
    fits[
        (fits['origin'] == origin) & (fits['data_selection'] == sel['alias_of'])
    ].assign(data_selection=key)
    for origin, selections in selections_by_origin.items()
    for key, sel in selections.items()
    if 'alias_of' in sel
]
fits = pd.concat([fits, *aliases], ignore_index=True)

# — phase 2: one row per configuration, the per-origin scores aggregated ———
def summarize_config(group):

```

```

first = group.iloc[0]
ok = group[group['error'].isna()]
latest = ok[ok['origin'] == LATEST_ORIGIN]
p, d, q = first['order']
P, D, Q, _ = first['seasonal_order']
errors = group['error'].dropna()
row = {
    'data_selection': first['data_selection'],
    'target_transform': first['target_transform'],
    'order': first['order'],
    'seasonal_order': first['seasonal_order'],
    'd': d,
    'D': D,
    'run_name': (
        f'SARIMAX({p},{d},{q})({P},{D},{Q})[{s}] '
        f'({first["data_selection"]} | {first["target_transform"]})'
    ),
    'converged': len(ok) == expected_origins[first['data_selection']],
    'error': errors.iloc[0] if len(errors) else None,
    'fit_seconds': float(group['fit_seconds'].sum()),
    'origin_mase': dict(zip(ok['origin'], ok['mase'])),
    **{
        key: float(latest[key].iloc[0]) if len(latest) else np.nan
        for key in ['aicc', 'aic', 'bic']
    },
}
per_origin = {r['origin']: {k: r[k] for k in METRICS} for _, r in ok.iterrows()}
row.update(aggregate(per_origin) if per_origin else {})
return row

configs_df = pd.DataFrame([
    summarize_config(group)
    for _, group in fits.groupby(
        ['data_selection', 'target_transform', 'order', 'seasonal_order'], sort=False
    )
])

def log_result(client, result):
    cfg = model_configs[
        (LATEST_ORIGIN, result['data_selection'], result['target_transform'])
    ]
    p, d, q = result['order']
    P, D, Q, _ = result['seasonal_order']
    exog_label = (
        '+' .join(cfg['exog_train'].columns) if cfg['exog_train'] is not None else ''
    )
    params = {
        'p': p,
        'd': d,
        'q': q,
        'P': P,
        'D': D,
        'Q': Q,
        's': s,

```

```

        'transform': result['target_transform'],
        'ast_pipeline': cfg['pipeline'],
        'boxcox_lambda': cfg['boxcox_lambda'] or 'none',
        'data_selection': result['data_selection'],
        'exog': exog_label,
        'n_train': len(cfg['series']),
        'n_val': HORIZON,
        'eval_split': 'validation',
        'origins': ', '.join(origin_label(o) for o in result['origin_mase']),
        'selection_metric': 'mean_seasonal_mase',
    }
    if result['error']:
        params['error'] = result['error']

    values = {
        **{key: result.get(key) for key in ['aicc', 'aic', 'bic', *METRICS]},
        **{key: result.get(key) for key in AGG_METRICS},
        **{
            f'mase_origin_{origin:%Y%m}': value
            for origin, value in result['origin_mase'].items()
        },
        'fit_seconds': result['fit_seconds'],
    }
    timestamp = int(time.time() * 1000)
    metrics = [
        mlflow.entities.Metric(key, float(value), timestamp, 0)
        for key, value in values.items()
        if value is not None and np.isfinite(value)
    ]

    run = client.create_run(experiment.experiment_id, run_name=result['run_name'])
    client.log_batch(
        run.info.run_id,
        metrics=metrics,
        params=[mlflow.entities.Param(k, str(v)) for k, v in params.items()],
    )
    client.set_terminated(
        run.info.run_id, status='FINISHED' if result['converged'] else 'FAILED'
    )

# each run is 3 HTTP calls; threads overlap the request latency (~3x faster)
N_LOG_THREADS = 8
MLFLOW_MAX_MASE = 1.5 # only configurations below this mean seasonal MASE are logged

client = mlflow.MlflowClient()
log_start = time.perf_counter()
records = configs_df[
    configs_df['converged'] & (configs_df['mase_mean'] < MLFLOW_MAX_MASE)
].to_dict('records')
with ThreadPoolExecutor(max_workers=N_LOG_THREADS) as executor:
    list(
        tqdm(
            executor.map(lambda result: log_result(client, result), records),
            total=len(records),
            desc='Logging to MLflow',

```

```

    )
    )
print(
    f'Logged {len(records)} of {len(configs_df)} configurations '
    f'(mase_mean < {MLFLOW_MAX_MASE}) in {time.perf_counter() - log_start:,.0f}s'
)

results_df = (
    configs_df.query('converged').sort_values('mase_mean').reset_index(drop=True)
)
results_df['beats_seasonal_naive'] = results_df['mase_mean'] < BASELINE_MASE

print(f'Converged at every origin: {len(results_df)} / {len(configs_df)}')
print(
    f'{results_df["beats_seasonal_naive"].sum()} models beat the seasonal naive '
    f'(mean seasonal MASE {BASELINE_MASE:.3f})'
)
results_df[['run_name', *AGG_METRICS, 'rmse', 'mape', 'r2', 'forecast_bias']].head(
    10
).round(3)

```

Fitting 17280 models on 8 workers...

```

[Parallel(n_jobs=8)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=8)]: Done   2 tasks      | elapsed:    4.2s
[Parallel(n_jobs=8)]: Done  56 tasks      | elapsed:    4.9s
[Parallel(n_jobs=8)]: Done 357 tasks      | elapsed:   13.0s
[Parallel(n_jobs=8)]: Done 536 tasks      | elapsed:   16.9s
[Parallel(n_jobs=8)]: Done 860 tasks      | elapsed:   24.7s
[Parallel(n_jobs=8)]: Done 1256 tasks     | elapsed:   32.4s
[Parallel(n_jobs=8)]: Done 1724 tasks     | elapsed:   42.9s
[Parallel(n_jobs=8)]: Done 2264 tasks     | elapsed:   53.8s
[Parallel(n_jobs=8)]: Done 2876 tasks     | elapsed:   1.2min
[Parallel(n_jobs=8)]: Done 3224 tasks     | elapsed:   1.4min
[Parallel(n_jobs=8)]: Done 3602 tasks     | elapsed:   1.6min
[Parallel(n_jobs=8)]: Done 4016 tasks     | elapsed:   1.8min
[Parallel(n_jobs=8)]: Done 4466 tasks     | elapsed:   2.0min
[Parallel(n_jobs=8)]: Done 5256 tasks     | elapsed:   2.4min
[Parallel(n_jobs=8)]: Done 5826 tasks     | elapsed:   2.9min
[Parallel(n_jobs=8)]: Done 6384 tasks     | elapsed:   3.6min
[Parallel(n_jobs=8)]: Done 6978 tasks     | elapsed:   4.4min
[Parallel(n_jobs=8)]: Done 7608 tasks     | elapsed:   5.2min
[Parallel(n_jobs=8)]: Done 8274 tasks     | elapsed:   5.8min
[Parallel(n_jobs=8)]: Done 8976 tasks     | elapsed:   6.3min
[Parallel(n_jobs=8)]: Done 9714 tasks     | elapsed:   6.8min
[Parallel(n_jobs=8)]: Done 10960 tasks    | elapsed:   7.3min
[Parallel(n_jobs=8)]: Done 11986 tasks    | elapsed:   8.2min
[Parallel(n_jobs=8)]: Done 12832 tasks    | elapsed:   9.3min
[Parallel(n_jobs=8)]: Done 13714 tasks    | elapsed:  10.0min
[Parallel(n_jobs=8)]: Done 14632 tasks    | elapsed:  10.4min
[Parallel(n_jobs=8)]: Done 16348 tasks    | elapsed:  11.2min
[Parallel(n_jobs=8)]: Done 17265 out of 17280 | elapsed: 11.6min remaining:    0.5s
[Parallel(n_jobs=8)]: Done 17280 out of 17280 | elapsed: 11.6min finished

```

Fitting done in 696s

Logging to MLflow: 100% | 299/299 [05:45<00:00, 1.16s/it]

Logged 299 of 6048 configurations (mase_mean < 1.5) in 346s
Converged at every origin: 5964 / 6048
1056 models beat the seasonal naive (mean seasonal MASE 1.786)

Out[8]:

	run_name	mase_mean	mase_worst	mase_post	n_origins	rmse	mape
0	SARIMAX(0,1,2)(0,1,1) [12] (last_4y_covid_recov...	0.992	1.258	1.137	4	103693.084	7.302
1	SARIMAX(2,1,2)(0,1,1) [12] (last_4y_covid_recov...	1.072	1.450	1.276	4	113038.836	8.490
2	SARIMAX(1,1,1)(0,1,1) [12] (last_4y_covid_recov...	1.137	1.465	1.452	4	113123.407	8.542
3	SARIMAX(1,1,2)(0,1,1) [12] (last_4y_covid_recov...	1.167	1.559	1.479	4	110813.695	8.109
4	SARIMAX(0,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.175	2.183	1.530	4	153634.495	14.138
5	SARIMAX(0,1,2)(0,1,1) [12] (last_4y_covid_exog ...	1.190	1.594	1.533	4	121969.346	9.530
6	SARIMAX(1,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.199	2.190	1.535	4	154985.786	14.134
7	SARIMAX(0,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.219	2.377	1.627	4	201791.935	13.711
8	SARIMAX(0,1,2)(1,1,1) [12] (last_4y_covid_recov...	1.229	2.220	1.564	4	156537.101	14.369
9	SARIMAX(1,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.237	2.344	1.612	4	200435.717	13.421

Results

In [9]:

```
palette = {  
    'last_14y_covid_exog': 'midnightblue',  
    'last_12y_covid_exog': 'steelblue',  
    'last_10y_covid_exog': 'slateblue',  
    'last_8y_covid_exog': 'mediumpurple',  
    'last_6y_covid_exog': 'orchid',  
    'last_4y_covid_exog': 'palevioletred',
```

```

        'last_14y_covid_recovery_exog': 'darkslategray',
        'last_12y_covid_recovery_exog': 'teal',
        'last_10y_covid_recovery_exog': 'cadetblue',
        'last_8y_covid_recovery_exog': 'mediumaquamarine',
        'last_6y_covid_recovery_exog': 'darkturquoise',
        'last_4y_covid_recovery_exog': 'lightseagreen',
        'post_covid': 'seagreen',
        'post_covid_recovery_exog': 'darkolivegreen',
    }
    BAR_METRICS = ['mase_mean', 'mase_worst', 'rmse', 'mape', 'r2', 'forecast_bias']

    best_per_selection = (
        results_df
        .sort_values('mase_mean')
        .groupby('data_selection', sort=False)
        .first()
        .reset_index()
        .sort_values('mase_mean')
        .reset_index(drop=True)
    )
    best_per_selection['label'] = best_per_selection['run_name'].str.replace(
        'SARIMAX', '', regex=False
    )
    bar_colors = [palette[ds] for ds in best_per_selection['data_selection'][::-1]]

    fig, axes = plt.subplots(2, 3, figsize=(22, 10), sharey=True)
    fig.suptitle(
        'Best SARIMAX Model per Data Selection (ranked by mean seasonal MASE) - '
        'dashed line = seasonal naive',
        fontsize=14,
        fontweight='bold',
    )

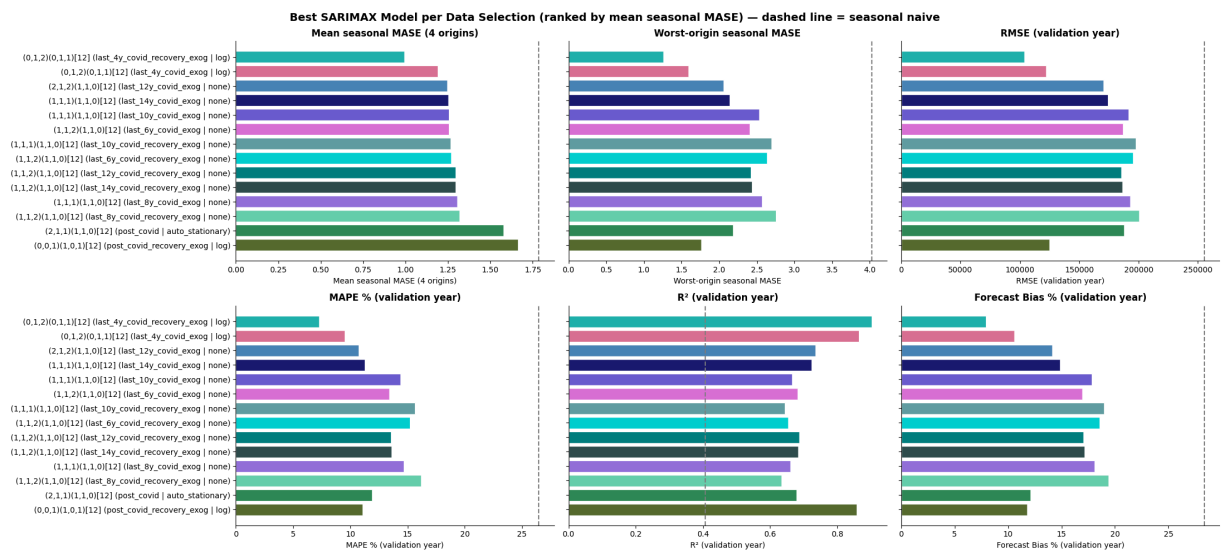
    for ax, col in zip(axes.flat, BAR_METRICS):
        title = METRIC_LABELS[col]
        ax.barh(
            best_per_selection['label'][::-1],
            best_per_selection[col][::-1],
            color=bar_colors,
            edgecolor='white',
        )
        ax.axvline(
            float(baselines.loc['seasonal_naive', col]),
            color='grey',
            linestyle='--',
        )
        if col == 'forecast_bias':
            ax.axvline(0, color='black', linewidth=0.8)
        ax.set_title(title, fontsize=12, fontweight='bold')
        ax.set_xlabel(title)
        sns.despine(ax=ax)

    plt.tight_layout()
    plt.show()

    display(

```

```
best_per_selection[
[
    'data_selection',
    'target_transform',
    'run_name',
    *AGG_METRICS,
    *METRICS,
    'beats_seasonal_naive',
]
)
```



	data_selection	target_transform	run_name	mase_mean	mase_wc
0	last_4y_covid_recovery_exog	log	SARIMAX(0,1,2)(0,1,1) [12] (last_4y_covid_recov...	0.992006	1.2570
1	last_4y_covid_exog	log	SARIMAX(0,1,2)(0,1,1) [12] (last_4y_covid_exog ...	1.189968	1.5940
2	last_12y_covid_exog	none	SARIMAX(2,1,2)(1,1,0) [12] (last_12y_covid_exog...	1.247833	2.0600
3	last_14y_covid_exog	none	SARIMAX(1,1,1)(1,1,0) [12] (last_14y_covid_exog...	1.252051	2.1390
4	last_10y_covid_exog	none	SARIMAX(1,1,1)(1,1,0) [12] (last_10y_covid_exog...	1.255705	2.5340
5	last_6y_covid_exog	none	SARIMAX(1,1,2)(1,1,0) [12] (last_6y_covid_exog ...	1.257337	2.4090
6	last_10y_covid_recovery_exog	none	SARIMAX(1,1,1)(1,1,0) [12] (last_10y_covid_reco...	1.266176	2.6930
7	last_6y_covid_recovery_exog	none	SARIMAX(1,1,2)(1,1,0) [12] (last_6y_covid_recov...	1.269365	2.6340
8	last_12y_covid_recovery_exog	none	SARIMAX(1,1,2)(1,1,0) [12] (last_12y_covid_reco...	1.294099	2.4230
9	last_14y_covid_recovery_exog	none	SARIMAX(1,1,2)(1,1,0) [12] (last_14y_covid_reco...	1.294133	2.4380
10	last_8y_covid_exog	none	SARIMAX(1,1,1)(1,1,0) [12] (last_8y_covid_exog ...	1.305186	2.5700
11	last_8y_covid_recovery_exog	none	SARIMAX(1,1,2)(1,1,0) [12] (last_8y_covid_recov...	1.317537	2.7550
12	post_covid	auto_stationary	SARIMAX(2,1,1)(1,1,0) [12] (post_covid auto_s...	1.578388	2.1870
13	post_covid_recovery_exog	log	SARIMAX(0,0,1)(1,0,1) [12] (post_covid_recovery...	1.663515	1.7660

Effect of Target Transformation and Differencing

Each cell summarizes all converged models that share the two choices on its axes (every remaining order) with the **p95 of the mean seasonal MASE**: the 5% quantile of the cell, the value that only the best 5% of its models beat. Unlike the single best model, it does not reward a combination where one order got lucky while its neighbours are much worse. The number under each value is how many models the cell summarizes.

These tables answer whether a transformation helps, and whether it replaces or still needs differencing. The best single models are in the charts above and in the top 10 below.

```
In [10]: P95_QUANTILE = 0.05 # p95: the value only the best 5% of a cell's models beat

def p95(values):
    return values.quantile(P95_QUANTILE)

def p95_pivot(index, columns):
    """p95 of the mean seasonal MASE per cell, and the number of models per cell."""
    kwargs = {'index': index, 'columns': columns, 'values': 'mase_mean'}
    return (
        results_df.pivot_table(**kwargs, aggfunc=p95),
        results_df.pivot_table(**kwargs, aggfunc='count'),
    )

def p95_labels(pivot, counts):
    """Cell labels: the p95 value and, below it, the number of models."""
    return pd.DataFrame(
        [
            [' if pd.isna(v) else f'{v:.3f}\n(n={int(n)})' for v, n in zip(values,
                                for values, ns in zip(pivot.to_numpy(), counts.to_numpy())
            ],
        index=pivot.index,
        columns=pivot.columns,
    )

p95_by_selection_transform, n_by_selection_transform = (
    frame.reindex(columns=TARGET_TRANSFORMS)
    for frame in p95_pivot('data_selection', 'target_transform')
)
p95_by_transform_diff, n_by_transform_diff = (
    frame.reindex(TARGET_TRANSFORMS)
    for frame in p95_pivot('target_transform', ['d', 'D'])
)

fig, axes = plt.subplots(1, 2, figsize=(18, 8), gridspec_kw={'width_ratios': [1.3,
for ax, pivot, counts, title, xlabel in [
    (
        axes[0],
```

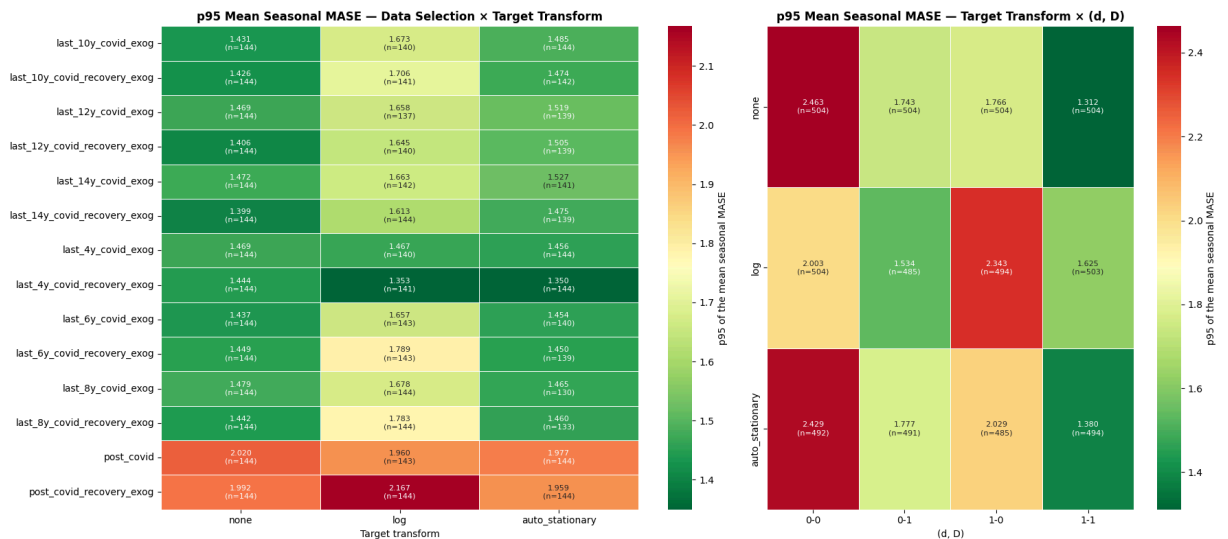
```

p95_by_selection_transform,
n_by_selection_transform,
'Data Selection x Target Transform',
'Target transform',
),
(
    axes[1],
    p95_by_transform_diff,
    n_by_transform_diff,
    'Target Transform x (d, D)',
    '(d, D)',
),
]:
    sns.heatmap(
        pivot,
        annot=p95_labels(pivot, counts),
        fmt='',
        annot_kws={'fontsize': 8},
        cmap='RdYlGn_r',
        linewidths=0.4,
        ax=ax,
        cbar_kws={'label': 'p95 of the mean seasonal MASE'},
    )
    ax.set_title(f'p95 Mean Seasonal MASE - {title}', fontweight='bold')
    ax.set_xlabel(xlabel)
    ax.set_ylabel('')

plt.tight_layout()
plt.show()

display(
    p95_by_selection_transform.assign(
        best=p95_by_selection_transform.idxmin(axis=1)
    ).round(3)
)
display(p95_by_transform_diff.round(3))

```



	target_transform	none	log	auto_stationary	best
data_selection					
	last_10y_covid_exog	1.431	1.673	1.485	none
	last_10y_covid_recovery_exog	1.426	1.706	1.474	none
	last_12y_covid_exog	1.469	1.658	1.519	none
	last_12y_covid_recovery_exog	1.406	1.645	1.505	none
	last_14y_covid_exog	1.472	1.663	1.527	none
	last_14y_covid_recovery_exog	1.399	1.613	1.475	none
	last_4y_covid_exog	1.469	1.467	1.456	auto_stationary
	last_4y_covid_recovery_exog	1.444	1.353	1.350	auto_stationary
	last_6y_covid_exog	1.437	1.657	1.454	none
	last_6y_covid_recovery_exog	1.449	1.789	1.450	none
	last_8y_covid_exog	1.479	1.678	1.465	auto_stationary
	last_8y_covid_recovery_exog	1.442	1.783	1.460	none
	post_covid	2.020	1.960	1.977	log
	post_covid_recovery_exog	1.992	2.167	1.959	auto_stationary

d **0** **1**
D **0** **1** **0** **1**

target_transform					
	none	2.463	1.743	1.766	1.312
	log	2.003	1.534	2.343	1.625
	auto_stationary	2.429	1.777	2.029	1.380

Best Model Diagnostics

```
In [11]: top10 = results_df.head(10).copy()

display(
    top10[
        [
            'data_selection',
            'run_name',
            *AGG_METRICS,
            'aicc',
            'aic',
            'bic',
        ]
    ]
)
```

```

    *METRICS,
    ]
    ]
)

```

	data_selection	run_name	mase_mean	mase_worst	mase_post	n_or
0	last_4y_covid_recovery_exog	SARIMAX(0,1,2)(0,1,1) [12] (last_4y_covid_recov...	0.992006	1.257627	1.137286	
1	last_4y_covid_recovery_exog	SARIMAX(2,1,2)(0,1,1) [12] (last_4y_covid_recov...	1.071647	1.449711	1.275819	
2	last_4y_covid_recovery_exog	SARIMAX(1,1,1)(0,1,1) [12] (last_4y_covid_recov...	1.136900	1.465201	1.451922	
3	last_4y_covid_recovery_exog	SARIMAX(1,1,2)(0,1,1) [12] (last_4y_covid_recov...	1.167229	1.558576	1.479311	
4	last_4y_covid_recovery_exog	SARIMAX(0,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.175090	2.182530	1.529951	
5	last_4y_covid_exog	SARIMAX(0,1,2)(0,1,1) [12] (last_4y_covid_exog ...	1.189968	1.594258	1.533209	
6	last_4y_covid_recovery_exog	SARIMAX(1,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.199021	2.190350	1.535128	
7	last_4y_covid_recovery_exog	SARIMAX(0,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.219302	2.377222	1.627297	
8	last_4y_covid_recovery_exog	SARIMAX(0,1,2)(1,1,1) [12] (last_4y_covid_recov...	1.228707	2.219643	1.564486	
9	last_4y_covid_recovery_exog	SARIMAX(1,1,1)(1,1,1) [12] (last_4y_covid_recov...	1.236524	2.344273	1.612089	

```

In [12]: def fit_model(row, configs):
    """Refit a configuration at the latest origin (trained up to the end of train).
    cfg = configs[(LATEST_ORIGIN, row['data_selection'], row['target_transform'])]
    return SARIMAX(
        cfg['series'],
        exog=cfg['exog_train'],
        order=row['order'],
        seasonal_order=row['seasonal_order'],
        enforce_stationarity=False,

```

```

        enforce_invertibility=False,
    ).fit(dis=False)

top10_fits = [fit_model(row, model_configs) for _, row in top10.iterrows()]

for i, (_, row) in enumerate(top10.iterrows()):
    print(f'#{i + 1:>2} [{row["data_selection"]} {row["run_name"]}']

# 1 [last_4y_covid_recovery_exog] SARIMAX(0,1,2)(0,1,1)[12] (last_4y_covid_recover
y_exog | log)
# 2 [last_4y_covid_recovery_exog] SARIMAX(2,1,2)(0,1,1)[12] (last_4y_covid_recover
y_exog | log)
# 3 [last_4y_covid_recovery_exog] SARIMAX(1,1,1)(0,1,1)[12] (last_4y_covid_recover
y_exog | log)
# 4 [last_4y_covid_recovery_exog] SARIMAX(1,1,2)(0,1,1)[12] (last_4y_covid_recover
y_exog | log)
# 5 [last_4y_covid_recovery_exog] SARIMAX(0,1,1)(1,1,1)[12] (last_4y_covid_recover
y_exog | auto_stationary)
# 6 [last_4y_covid_exog] SARIMAX(0,1,2)(0,1,1)[12] (last_4y_covid_exog | log)
# 7 [last_4y_covid_recovery_exog] SARIMAX(1,1,1)(1,1,1)[12] (last_4y_covid_recover
y_exog | auto_stationary)
# 8 [last_4y_covid_recovery_exog] SARIMAX(0,1,1)(1,1,1)[12] (last_4y_covid_recover
y_exog | none)
# 9 [last_4y_covid_recovery_exog] SARIMAX(0,1,2)(1,1,1)[12] (last_4y_covid_recover
y_exog | auto_stationary)
#10 [last_4y_covid_recovery_exog] SARIMAX(1,1,1)(1,1,1)[12] (last_4y_covid_recover
y_exog | none)

```

Residual Diagnostics

```

In [13]: fig, axes = plt.subplots(10, 3, figsize=(16, 40))
fig.suptitle(
    'Residual Diagnostics – Top 10 Models', fontsize=14, fontweight='bold', y=1.001
)

x_qq = np.linspace(-4, 4, 200)

for i, (_, row), fit in enumerate(zip(top10.iterrows(), top10_fits)):
    std_resid = fit.filter_results.standardized_forecasts_error[0]
    std_resid = std_resid[~np.isnan(std_resid)]

    row_label = f'#{i + 1} {row["run_name"].replace("SARIMAX", "")}'

    # Standardized residuals over time
    axes[i, 0].plot(std_resid, linewidth=1.2, color=palette[row['data_selection']])
    axes[i, 0].axhline(0, color='black', linestyle='--', alpha=0.5)
    axes[i, 0].set_title(f'{row_label}\nStandardized Residuals', fontsize=8)
    axes[i, 0].set_xlabel('Observation', fontsize=8)
    sns.despine(ax=axes[i, 0])

    # Histogram + KDE + N(0,1)
    axes[i, 1].hist(
        std_resid,
        bins='auto',

```

```

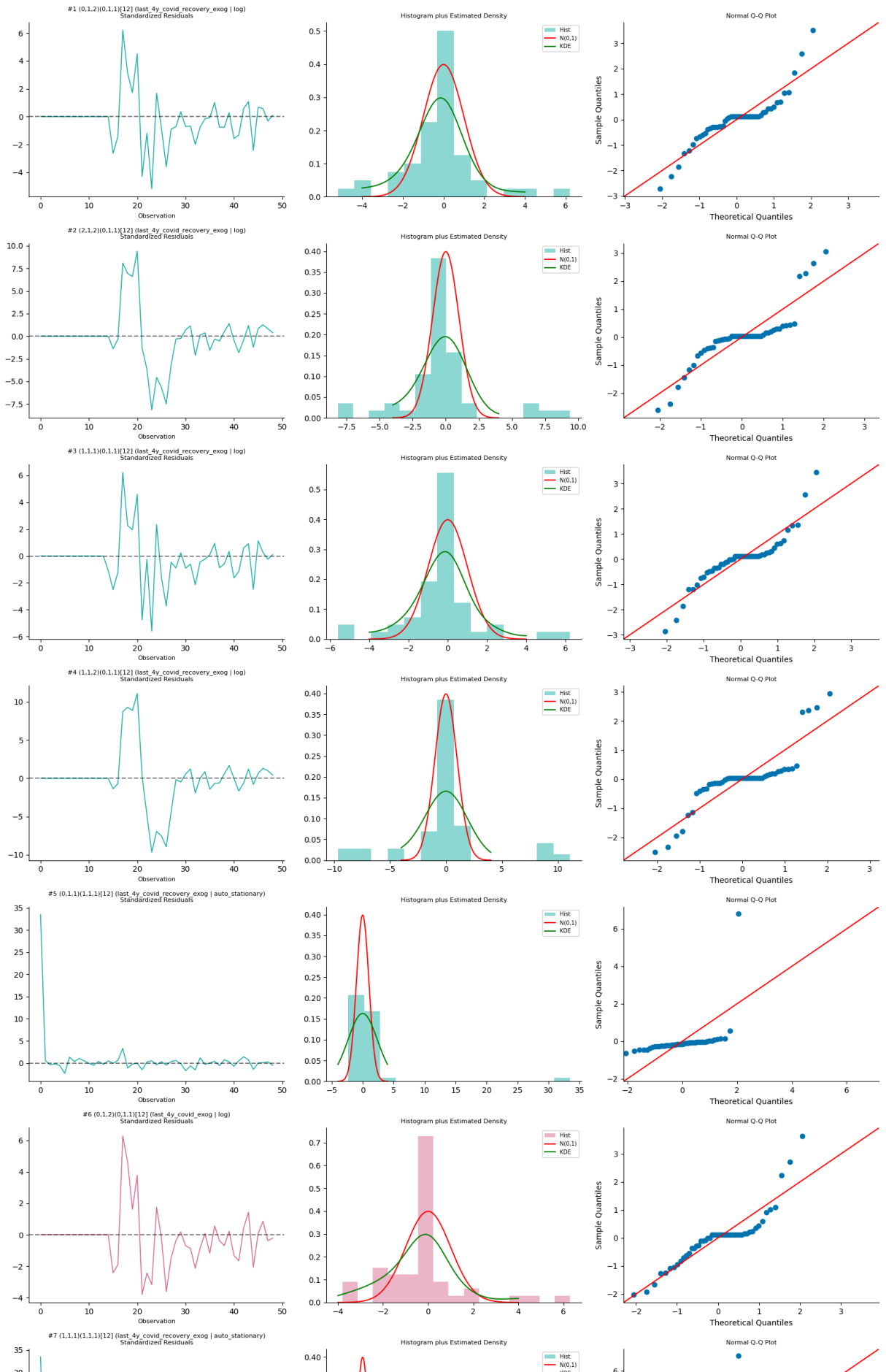
        density=True,
        alpha=0.5,
        color=palette[row['data_selection']],
        label='Hist',
    )
    axes[i, 1].plot(x_qq, stats.norm.pdf(x_qq, 0, 1), 'r-', label='N(0,1)')
    kde = stats.gaussian_kde(std_resid)
    axes[i, 1].plot(x_qq, kde(x_qq), 'g-', label='KDE')
    axes[i, 1].set_title('Histogram plus Estimated Density', fontsize=8)
    axes[i, 1].legend(fontsize=7)
    sns.despine(ax=axes[i, 1])

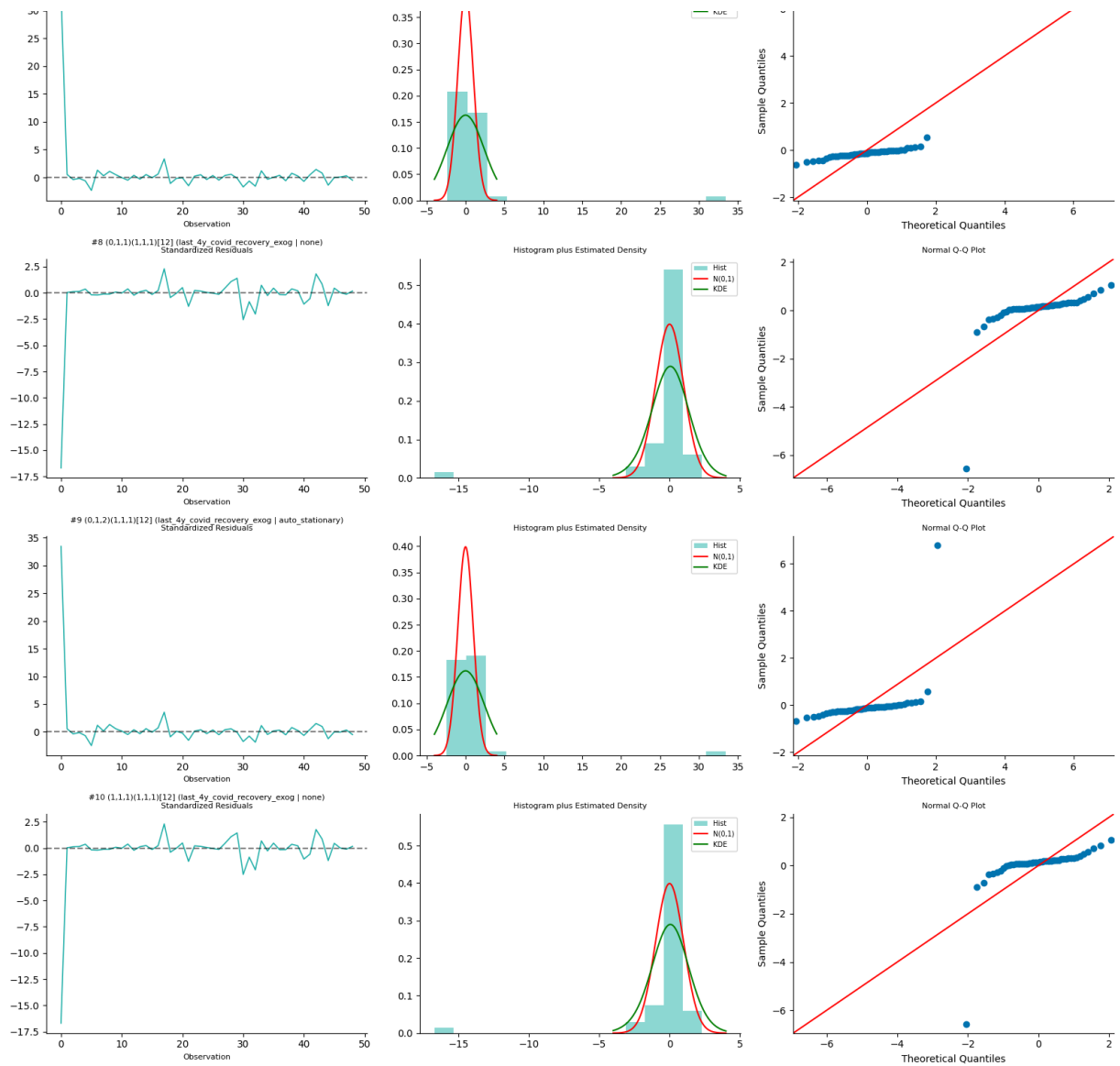
    # Normal Q-Q plot
    sm.qqplot(std_resid, line='45', fit=True, ax=axes[i, 2])
    axes[i, 2].set_title('Normal Q-Q Plot', fontsize=8)
    sns.despine(ax=axes[i, 2])

plt.tight_layout()
plt.show()

```

Residual Diagnostics — Top 10 Models





Final Result

```
In [14]: fig, axes = plt.subplots(5, 2, figsize=(14, 24))
fig.suptitle(
    f'Top 10 Models - Validation Set Forecasts '
    f'({val_monthly.index[0]:%b %Y}-{val_monthly.index[-1]:%b %Y})',
    fontsize=15,
    fontweight='bold',
)

for i, (ax, (_, row), fit) in enumerate(zip(axes.flat, top10.iterrows(), top10_fits)):
    cfg = model_configs[(LATEST_ORIGIN, row['data_selection'], row['target_transform'])]
    forecast = forecast_arrivals(fit, cfg)
    train_plot = cfg['raw_series'].iloc[-24:]

    ax.plot(
        train_plot.index,
        train_plot.values,
        label='Train (last 24 m)',
        linewidth=1.5,
```

```

        color=palette[row['data_selection']],
        alpha=0.7,
    )
    ax.plot(
        val_monthly.index,
        val_monthly['arrival_count'],
        label='Actual',
        linewidth=2,
        color='black',
    )
    ax.plot(
        val_monthly.index,
        forecast,
        label='Forecast',
        linewidth=2,
        linestyle='--',
        color='tomato',
        marker='o',
        markersize=5,
    )

    ax.set_title(
        f'#{i + 1} {row["run_name"].replace("SARIMAX", "")}\n'
        f'MASE mean={row["mase_mean"]:.3f} | validation={row["mase"]:.3f}\n'
        f'RMSE={row["rmse"]:.0f} MAPE={row["mape"]:.1f}% R²={row["r2"]:.3f}',
        fontsize=10,
        fontweight='bold',
    )
    ax.set_xlabel('Date', fontsize=9)
    ax.set_ylabel('Monthly Arrivals', fontsize=9)
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x / 1000)}k')
    ax.tick_params(axis='both', labelsize=8)
    ax.legend(fontsize=8)
    sns.despine(ax=ax)

plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.show()

```

Top 10 Models — Validation Set Forecasts (Sep 2024-Aug 2025)

